

Questões

Todas as questões são incrementais

1. Crie uma classe chamada `Planet` que represente um planeta do sistema solar. A classe deve ter os seguintes atributos:

- `name` – uma string representando o nome do planeta.
- `mass` – um número ponto flutuante representando a massa do planeta em quilogramas
- `diameter` – um número ponto flutuante representando o diâmetro do planeta em quilômetros.
- `distance_from_sun` – um número ponto flutuante representando a distância do planeta em relação ao sol em quilômetros.

Crie um método `__init__` para inicializar esses atributos.

Crie dois planetas da classe.

```
# Exemplo
earth = Planet("Terra", 6 * 10**24, 12700, 150 * 10**6)
mars = Planet("Marte", 0.6 * 10**24, 6800, 230 * 10**6)
```

2. Adicione a sua classe do exercício (1), um método `describe`, que imprima os detalhes do Planeta. Use os dois planetas criados anteriormente para testar a implementação do método.

```
# Exemplo
earth = Planet("Terra", 6 * 10**24, 12700, 150 * 10**6)
mars = Planet("Marte", 0.6 * 10**24, 6800, 230 * 10**6)

earth.describe()
print()
mars.describe()
```

3. Substitua o método `describe` do exercício anterior pelo método especial `__str__`. O resultado deve permanecer o mesmo.

```
# Exemplo
earth = Planet("Terra", 6 * 10**24, 12700, 150 * 10**6)
mars = Planet("Marte", 0.6 * 10**24, 6800, 230 * 10**6)

print(earth)
print()
print(mars)
```

4. Crie um método para calcular e retornar o período orbital do planeta. Use a fórmula:

$$T = 2 \cdot \pi \sqrt{\frac{R^3}{GM}}$$

G: constante de gravitação universal ($\cong 6.674 \cdot 10^{-11} m^3 kg^{-1} s^{-1}$)

M: massa do Sol em quilogramas

R: distância do planeta ao Sol em metros

Declare constantes internas à classe para representar M e G. Atenção às unidades.

```
# Exemplo
earth = Planet("Terra", 6 * 10**24, 12700, 150 * 10**6)
mars = Planet("Marte", 0.6 * 10**24, 6800, 230 * 10**6)

print(earth)
print(f"Período Orbital da Terra: {earth.orbital_period():.2f} anos")
print()
print(mars)
print(f"Período Orbital de Marte: {mars.orbital_period():.2f} anos")
```

5. Crie uma classe Moon que represente uma lua de um Planeta. A classe deve ter os seguintes atributos:

- name – uma string representando o nome da lua.
- diameter – um número ponto flutuante representando o diâmetro da lua em quilômetros.
- distance_from_planet – um número ponto flutuante representando a distância média da lua em relação ao Planeta em quilômetros.

Crie um método `__init__` para inicializar esses atributos e crie um método `__str__`, que imprima os detalhes da Lua.

Atualize a classe Planet:

- Adicione um novo atributo à classe Planet denominado moons, que deve ser uma lista de objetos do tipo Moon.
- Adicione um método `add_moon( self, moon)` que receba um objeto Moon como parâmetro e o adicione à lista de luas (moons).

- Crie um método `describe_moons`, que chame o método `__str__` de cada uma das suas luas.

```
# Exemplo
earth = Planet("Terra", 6 * 10**24, 12700, 150 * 10**6)
moon_earth = Moon("Lua", 3475, 384400)
earth.add_moon(moon_earth)

mars = Planet("Marte", 0.6 * 10**24, 6800, 230 * 10**6)
fobos_mars = Moon("Fobos", 22.4, 9377)
deimos_mars = Moon("Deimos", 12.4, 23460)
mars.add_moon(fobos_mars)
mars.add_moon(deimos_mars)

print(earth)
earth.describe_moons()
print(f"Período Orbital da Terra: {earth.orbital_period():.2f} anos")

print()

print(mars)
mars.describe_moons()
print(f"Período Orbital de Marte: {mars.orbital_period():.2f} anos")
```

6. Crie uma classe `SolarSystem` que represente um sistema solar. A classe deve ter como atributo:

- `planets`: Uma lista de objetos do tipo `Planet`.

A classe deve ter métodos:

- `add_planet(self, planet)`: adicionar um objeto `Planet` à lista de planetas (`planets`).
- `describe(self, name=0)`: se for fornecido um nome, descreve os detalhes do planeta específico com o nome. Caso contrário, escreve os detalhes de todos os planetas.
- `total_planets(self)`: retornar o número de planetas no sistema solar.
- `total_mass(self)`: calcular a massa total do sistema solar

```
# Exemplo
solar_system = SolarSystem()

earth = Planet("Terra", 6 * 10**24, 12700, 150 * 10**6)
moon_earth = Moon("Lua", 3475, 384400)
earth.add_moon(moon_earth)
solar_system.add_planet(earth)
```

```

mars = Planet("Marte", 0.6 * 10**24, 6800, 230 * 10**6)
fobos_mars = Moon("Fobos", 22.4, 9377)
deimos_mars = Moon("Deimos", 12.4, 23460)
mars.add_moon(fobos_mars)
mars.add_moon(deimos_mars)
solar_system.add_planet(mars)

# Descreve todos os planetas
print(solar_system.describe())

print()

# Descreve especificamente a Terra
print(solar_system.describe("Terra"))

print(f"Numero total de planetas: {solar_system.total_planets()}")
print(f"Massa total do sistema solar: {solar_system.total_mass():.2e}
kg")

```

7. Modifique o programa para permitir que o usuário insira interativamente os dados dos planetas e suas luas. O programa deve:

- Permitir que o usuário adicione múltiplos planetas ao sistema solar.
- Para cada planeta, permitir que o usuário adicione uma ou mais luas.
- Exibir uma descrição de todos os planetas e suas respectivas luas no sistema solar.
- Permitir que o usuário solicite a descrição detalhada de um planeta específico.
- Exibir o número total de planetas e a massa total do sistema solar.